

Νευρωνικά Δίκτυα - Perceptron

3η Εργαστηριακή Άσκηση
Τεχνητή Νοημοσύνη

Βασίλειος Μπίτζας 1083796 4ο

Τμήμα Μηχανικών Η/Υ & Πληροφορικής
Πανεπιστήμιο Πατρών

Περιεχόμενα

1	Ανάλυση Υλοποίησης Perceptron	1
1.1	Εισαγωγή	1
1.2	Κλάση Perceptron	1
1.2.1	Δομή και Μεταβλητές	1
1.2.2	Αναλυτική Λειτουργία της Μεθόδου Εκπαίδευσης (fit)	2
1.2.3	Λεπτομερής Περιγραφή της Μεθόδου Πρόβλεψης (predict)	2
1.3	Βοηθητικές Συναρτήσεις	3
1.3.1	Συνάρτηση linspace	3
1.3.2	Συνάρτηση load_data	3
1.3.3	Συνάρτηση get_min_max	4
1.3.4	Συναρτήσεις Οπτικοποίησης Τερματικού	5
1.3.5	Διαχείριση Οπτικοποίησης	6
1.4	Μαθηματικό Υπόβαθρο	6
1.4.1	Συνάρτηση Απόφασης	6
1.4.2	Αλγόριθμος Εκπαίδευσης	6
1.5	Χρήση του Κώδικα	7
1.6	Συμπεράσματα	7
2	Αποτελέσματα για τα δοθέντα δεδομένα	8
2.1	Ανάλυση Εκπαίδευσης	8
2.2	Αποτελέσματα Ταξινόμησης	8
2.3	Ερμηνεία Αποτελεσμάτων	8
2.4	Οπτικοποίηση Αποτελεσμάτων	9

Κεφάλαιο 1

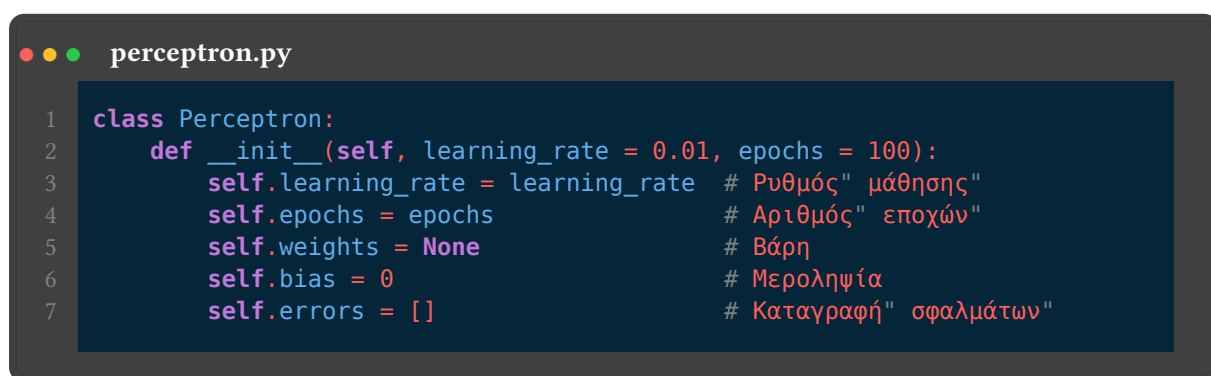
Ανάλυση Υλοποίησης Perceptron

1.1 Εισαγωγή

Η παρούσα υλοποίηση αφορά ένα απλό perceptron, το οποίο είναι ένα από τους βασικότερους τύπους τεχνητών νευρώνων. Το perceptron χρησιμοποιείται για δυαδική ταξινόμηση, διαχωρίζοντας τα δεδομένα σε δύο κλάσεις.

1.2 Κλάση Perceptron

1.2.1 Δομή και Μεταβλητές



```
1 class Perceptron:
2     def __init__(self, learning_rate = 0.01, epochs = 100):
3         self.learning_rate = learning_rate # Ρυθμός " μάθησης "
4         self.epochs = epochs # Αριθμός " εποχών "
5         self.weights = None # Βάρη
6         self.bias = 0 # Μεροληψία
7         self.errors = [] # Καταγραφή " σφαλμάτων "
```

Η κλάση Perceptron περιλαμβάνει τις εξής βασικές μεταβλητές:

- **learning_rate**: Καθορίζει το μέγεθος του βήματος προσαρμογής των βαρών
- **epochs**: Ορίζει τον αριθμό επαναλήψεων της εκπαίδευσης
- **weights**: Αποθηκεύει τα βάρη του perceptron
- **bias**: Αποθηκεύει τη μεροληψία του perceptron
- **errors**: Καταγράφει τα σφάλματα ανά εποχή

1.2.2 Αναλυτική Λειτουργία της Μεθόδου Εκπαίδευσης (fit)

```

1 def fit(self, X, y):
2     n_features = len(X[0])
3     self.weights = [0.0] * n_features
4     self.bias = 0

```

Η μέθοδος *fit()* λειτουργεί ως εξής:

1. Αρχικοποίηση:

- Δημιουργεί διάνυσμα βαρών μηδενικών τιμών με μέγεθος ίσο με τον αριθμό χαρακτηριστικών
- Αρχικοποιεί τη μεροληψία (bias) στο μηδέν

2. Διαδικασία Εκπαίδευσης:

- Επαναλαμβάνει για κάθε εποχή:
 - Υπολογίζει την έξοδο για κάθε δείγμα εκπαίδευσης
 - Συγκρίνει την πρόβλεψη με την πραγματική τιμή
 - Ενημερώνει τα βάρη και τη μεροληψία όταν υπάρχει σφάλμα

3. Ενημέρωση Βαρών: Όταν η πρόβλεψη είναι λανθασμένη:

```

1 update = self.learning_rate * y[i]
2 self.weights = [w + update * x for w, x in zip(self.weights, X[i])]
3 self.bias += update

```

1.2.3 Λεπτομερής Περιγραφή της Μεθόδου Πρόβλεψης (predict)

```

1 def predict(self, X):

```

Η μέθοδος *predict()* εκτελεί:

1. Υπολογισμός Γραμμικής Εξόδου:

- Πολλαπλασιάζει κάθε χαρακτηριστικό με το αντίστοιχο βάρος
- Προσθέτει τη μεροληψία
- Εφαρμόζει τη συνάρτηση ενεργοποίησης *sign()*

2. Επιστροφή Προβλέψεων:

- Επιστρέφει λίστα με τις προβλέψεις (-1 ή +1)

1.3 Βοηθητικές Συναρτήσεις

1.3.1 Συνάρτηση linspace

```
perceptron.py
1 def linspace(start, stop, num):
2     """Αντικατάσταση της numpy.linspace"""
3     if num == 1:
4         return [start]
5     step = (stop - start) / (num - 1)
6     return [start + i * step for i in range(num)]
```

Η συνάρτηση **linspace** υλοποιεί την αντίστοιχη λειτουργικότητα της **numpy.linspace**:

- **Είσοδος:**
 - **start**: Αρχική τιμή του διαστήματος
 - **stop**: Τελική τιμή του διαστήματος
 - **num**: Αριθμός σημείων
- **Έξοδος:** Λίστα με **num** ισαπέχοντα σημεία
- **Χρήση:** Χρησιμοποιείται για τη δημιουργία σημείων στη γραφική απεικόνιση

1.3.2 Συνάρτηση load_data

```
perceptron.py
1 def load_data(filename):
2     X, y = [], []
3     with open(filename, 'r') as f:
4         next(f) # Παράλειψη "κεφαλίδας"
5         for line in f:
6             x1, x2, label = map(float, line.strip().split(','))
7             X.append([x1, x2])
8             y.append(1 if label == 1.0 else -1)
9     return X, y
```

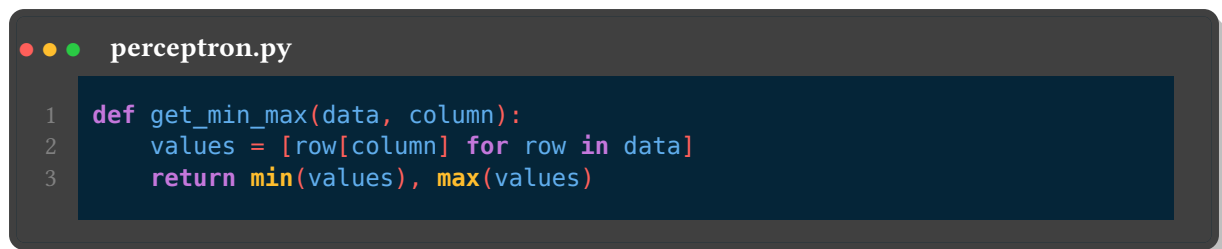
Η συνάρτηση **load_data** διαχειρίζεται την ανάγνωση και επεξεργασία των δεδομένων:

- **Λειτουργία:**
 - Διαβάζει αρχείο CSV με δεδομένα εκπαίδευσης ή ελέγχου
 - Παραλείπει την πρώτη γραμμή (κεφαλίδες)
 - Μετατρέπει τις τιμές σε αριθμούς κινητής υποδιαστολής
 - Διαχωρίζει τα χαρακτηριστικά (X) από τις ετικέτες (y)

- **Επιστρεφόμενες τιμές:**

- **X**: Λίστα με ζεύγη τιμών $[x_1, x_2]$
- **y**: Λίστα με ετικέτες κλάσεων (+1 ή -1)

1.3.3 Συνάρτηση `get_min_max`



```
perceptron.py
1 def get_min_max(data, column):
2     values = [row[column] for row in data]
3     return min(values), max(values)
```

Η βοηθητική συνάρτηση `get_min_max`:

- **Σκοπός:** Εύρεση ελάχιστης και μέγιστης τιμής συγκεκριμένης στήλης δεδομένων
- **Χρήση:**
 - Καθορισμός ορίων γραφικής παράστασης
 - Υπολογισμός εύρους τιμών για οπτικοποίηση
- **Παράμετροι:**
 - **data**: Λίστα με δεδομένα
 - **column**: Δείκτης στήλης για ανάλυση

1.3.4 Συνάρτησεις Οπτικοποίησης Τερματικού

Συνάρτηση `print_decision_boundary`

```

1 def print_decision_boundary(X, y, perceptron, width=50, height=20):
2     """Οπτικοποίηση ορίου απόφασης στο τερματικό"""
3     x_min, x_max = get_min_max(X, 0)
4     y_min, y_max = get_min_max(X, 1)
5
6     print("\nΌριο Απόφασης:")
7     print("-" * (width + 2))
8
9     for i in range(height):
10        y_val = y_max - (i * (y_max - y_min) / height)
11        line = "|"
12        for j in range(width):
13            x_val = x_min + (j * (x_max - x_min) / width)
14            prediction = 1 if (perceptron.weights[0] * x_val +
15                             perceptron.weights[1] * y_val +
16                             perceptron.bias) >= 0 else -1
17            line += "+" if prediction == 1 else "-"
18        line += "|"
19        print(line)
20    print("-" * (width + 2))
21    print(f"Υπόμνημα: '+' = θετική περιοχή , '-' = αρνητική περιοχή ")

```

Η συνάρτηση `print_decision_boundary` παρέχει μια εναλλακτική απεικόνιση του ορίου απόφασης στο τερματικό:

- Δημιουργεί ASCII απεικόνιση του χώρου χαρακτηριστικών
- Χρησιμοποιεί "+" για θετική περιοχή και "-" για αρνητική
- Προσαρμόζεται στο μέγεθος του τερματικού (προεπιλογή: 50×20)

Συνάρτηση `print_error_plot`

```

1 def print_error_plot(errors):
2     """Οπτικοποίηση εξέλιξης σφαλμάτων στο τερματικό"""
3     print("Εξέλιξη Σφαλμάτων:")
4     max_error = max(errors)
5     width = 40
6     for i, error in enumerate(errors, 1):
7         bar_length = int((error / max_error) * width)
8         print(f"Εποχή {i:2d}: █{' ' * bar_length} ({error})")

```

Η συνάρτηση `print_error_plot` παρέχει:

- Γραφική αναπαράσταση σφαλμάτων με μπάρες ASCII
- Κλιμάκωση σε σχέση με το μέγιστο σφάλμα

- Αριθμητική τιμή σφάλματος για κάθε εποχή

1.3.5 Διαχείριση Οπτικοποίησης

Το πρόγραμμα υποστηρίζει δύο τρόπους οπτικοποίησης: Ελέγχει τη διαθεσιμότητα των βιβλιοθηκών οπτικοποίησης:

```

perceptron.py
1 try:
2     import matplotlib.pyplot as plt
3     import mplcursors
4     VISUALIZATION_AVAILABLE = True
5 except ImportError:
6     VISUALIZATION_AVAILABLE = False
7     print("Προειδοποίηση: Οι βιβλιοθήκες οπτικοποίησης "
8           "δεν βρέθηκαν . Χρήση εξόδου τερματικού .\n")

```

1. Γραφική Απεικόνιση (με matplotlib):

- Δημιουργεί διαδραστικά γραφήματα όταν είναι διαθέσιμη η matplotlib
- Περιλαμβάνει όλες τις λειτουργίες που αναφέρθηκαν προηγουμένως

2. Απεικόνιση Τερματικού (εναλλακτική):

- Ενεργοποιείται αυτόματα όταν δεν υπάρχουν οι βιβλιοθήκες γραφικών
- Παρέχει ASCII απεικονίσεις του ορίου απόφασης και των σφαλμάτων
- Προσφέρει εύκολα κατανοητή οπτικοποίηση χωρίς εξωτερικές εξαρτήσεις

1.4 Μαθηματικό Υπόβαθρο

1.4.1 Συνάρτηση Απόφασης

Η μαθηματική έκφραση του perceptron είναι:

$$f(x) = \text{sign}\left(\sum (w_i x_i) + b\right)$$

όπου:

- w_i : τα βάρη του perceptron
- x_i : τα χαρακτηριστικά εισόδου
- b : η μεροληψία
- $\text{sign}()$: η συνάρτηση πρόσημου που επιστρέφει +1 ή -1

1.4.2 Αλγόριθμος Εκπαίδευσης

Για κάθε δείγμα εκπαίδευσης:

1. Υπολογισμός Εξόδου:

$$y_{pred} = \text{sign}(w_1x_1 + w_2x_2 + b)$$

2. Ενημέρωση Βαρών:

$$w_{new} = w_{old} + \eta \cdot (y_{true} - y_{pred}) \cdot x$$

όπου η είναι ο ρυθμός μάθησης

3. Ενημέρωση Μεροληψίας:

$$b_{new} = b_{old} + \eta \cdot (y_{true} - y_{pred})$$

1.5 Χρήση του Κώδικα

1. Προετοιμασία δεδομένων σε μορφή CSV χωρισμένων με κόμμα
2. Φόρτωση δεδομένων εκπαίδευσης (*training_data.csv*) και ελέγχου (*test_data.csv*)
3. Δημιουργία και εκπαίδευση του perceptron
4. Αξιολόγηση απόδοσης στο σετ ελέγχου
5. Οπτικοποίηση αποτελεσμάτων

1.6 Συμπεράσματα

Η υλοποίηση παρέχει μια πλήρη εικόνα της λειτουργίας του perceptron, συμπεριλαμβανομένης της εκπαίδευσης, της αξιολόγησης και της οπτικοποίησης. Η διαδραστική απεικόνιση βοηθά στην κατανόηση του τρόπου με τον οποίο το perceptron διαχωρίζει τα δεδομένα στο χώρο των χαρακτηριστικών.

Κεφάλαιο 2

Αποτελέσματα για τα δοθέντα δεδομένα

2.1 Ανάλυση Εκπαίδευσης

Η εκπαίδευση του perceptron έδειξε εξαιρετική σύγκλιση:

- **Αρχική Εποχή:** Μόνο 1 λάθος στην πρώτη εποχή
- **Σύγκλιση:** Πλήρης σύγκλιση (μηδέν λάθη) από τη δεύτερη εποχή
- **Σταθερότητα:** Διατήρηση μηδενικών λαθών για τις υπόλοιπες 18 εποχές

2.2 Αποτελέσματα Ταξινόμησης

Μετά την εκπαίδευση, το perceptron πέτυχε:

- **Ακρίβεια Ελέγχου:** 96.00% στο σύνολο δεδομένων ελέγχου
- **Όριο Απόφασης:** Η εξίσωση του ορίου απόφασης με ρυθμό μάθησης 0.1 και 20 εποχές είναι:

$$0.25x_1 + 0.29x_2 - 0.10 = 0$$

2.3 Ερμηνεία Αποτελεσμάτων

Τα αποτελέσματα υποδεικνύουν:

1. Ποιότητα Εκπαίδευσης:

- Ταχεία σύγκλιση (μόλις σε 2 εποχές)
- Εξαιρετική σταθερότητα κατά την εκπαίδευση
- Αποτελεσματική εκμάθηση του διαχωρισμού των κλάσεων

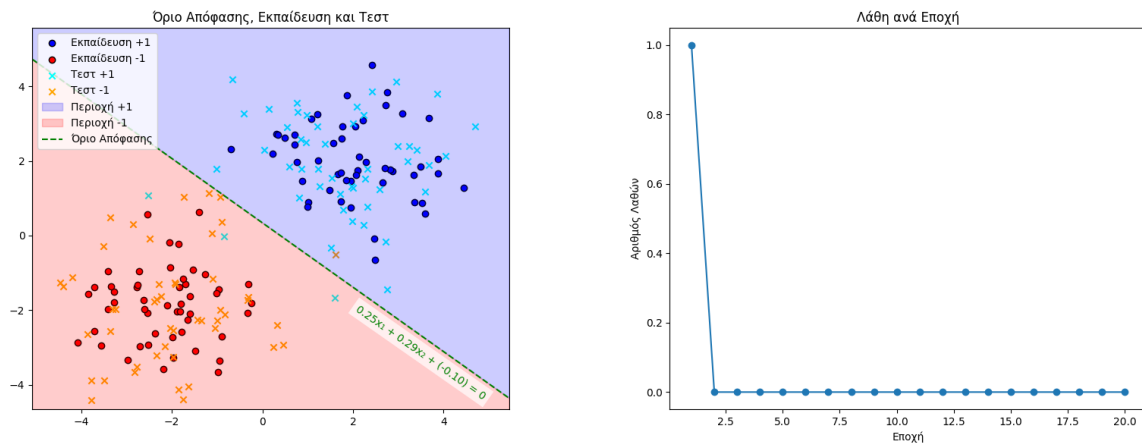
2. Γενίκευση:

- Η υψηλή ακρίβεια (96%) στο σύνολο ελέγχου δείχνει εξαιρετική ικανότητα γενίκευσης
- Ελάχιστη υπερπροσαρμογή στα δεδομένα εκπαίδευσης

3. Χαρακτηριστικά Ορίου Απόφασης:

- Παρόμοια βάρη για τα x_1 (0.25) και x_2 (0.29) υποδεικνύουν ισοδύναμη σημασία των χαρακτηριστικών
- Μικρή τιμή μεροληψίας (-0.10) υποδηλώνει ότι το όριο απόφασης περνά κοντά στην αρχή των αξόνων

2.4 Οπτικοποίηση Αποτελεσμάτων



(α') Απεικόνιση δεδομένων εκπαίδευσης, ελέγχου και ορίου απόφασης

(β') Γράφημα σφαλμάτων ανά εποχή εκπαίδευσης

Σχήμα 2.1: Οπτικοποίηση αποτελεσμάτων εκπαίδευσης του Perceptron

Στην Εικόνα 2.1 παρουσιάζονται τα αποτελέσματα της εκπαίδευσης:

- Εικόνα 2.1α': Σημεία εκπαίδευσης (μπλε/κόκκινο) και ελέγχου (κυανό/πορτοκαλί), καθώς και το όριο απόφασης (πράσινη διακεκομμένη γραμμή) που υπολόγισε το perceptron
- Εικόνα 2.1β': Ταχεία σύγκλιση του αλγορίθμου, με μόνο ένα σφάλμα στην πρώτη εποχή και μηδενικά σφάλματα στη συνέχεια